生成对抗网络（GAN）数学原理与 Chinese_Flower 项目深度解析

严格遵循「数据科学六步标准流程」与「机器学习五步标准流程」：从课堂白板（线性回归最小二乘、逻辑回归 MLE）到 CycleGAN 源码精讲

作者 / 整理：结合课堂板书/课件、机器学习数学理论与 Chinese_Flower 源码整理

日期：2026-10-25

目录

导读：如何用「数据科学标准流程」与「机器学习标准流程」贯通课堂数学、GAN 原理与 Chinese_Flower 项目？ 4

第 1 章 · 数据科学六步标准流程全景（对应课堂课件 1.1–1.3 与项目架构映射） 5

1.1 数据科学标准流水线（1.1 Pipeline）与 Chinese_Flower 全模块映射 5

1.2 课堂课件 1.2 & 1.3 核心规范完整梳理 6

1.2.1 七大常见数据抽取技术（1.2 Data Extraction） 6

1.2.2 数据预处理三大分支与问题解决方案（1.3 Data Pre-Processing） 6

第 2 章 · 机器学习五步标准流程与数学基础（课堂三张白板完整还原） 7

2.1 白板一精讲：线性变换、线性回归与最小二乘（LS）正交投影推导 8

Step 1：数据集与线性变换空间定义（Dataset & Linear Transformation） 8

Step 2：线性回归模型参数化（Model Parameterization） 8

Step 3：超定方程组困境与二范数平方损失构造（Loss Construction） 8

Step 4：最小二乘（Least Squares, LS）几何正交投影与正规方程推导 9

2.2 白板二与白板三精讲：逻辑回归（Logistic Regression）与极大似然估计（MLE） 9

Step 1：二分类数据集定义（Binary Classification Dataset） 10

Step 2：Sigmoid 激活函数与后验概率参数化（Model Parameterization） 10

Step 3：伯努利分布合成与极大似然估计（MLE "Log it"） 10

Step 4：白板三的负对数似然最小化（Minimization）与$\{-1,+1\}$ 标签统一形式 10

Step 5：基于预训练模型的预测与阈值推理（Prediction / Inference） 11

2.3 生成模型必备的信息论与深度学习数学工具 11

2.3.1 KL 散度、吉布斯不等式证明与 JS 散度 12

2.3.2 对抗训练中的复合函数链式法则与 Adam 优化器 12

2.3.3 卷积、转置卷积尺寸公式与 InstanceNorm2d 数学原理 13

第 3 章 · 按「机器学习五步标准流程」推导 GAN 家族与 CycleGAN 原理 13

3.1 原始 GAN（Vanilla GAN）五步标准建模与变分推导 13

Step 1 & Step 2：分布定义与双网络参数化 13

Step 3：直接由白板二逻辑回归 MLE 导出极小极大博弈目标$V(D,G)$ 14

Step 4：变分极值推导（证明最优判别器D_G^* 与 JS 散度等价性） 14

Step 5：纳什均衡与白板三决策阈值$P_{\text{Threshold}}=0.5$ 的印证 14

3.2 最小二乘 GAN（LSGAN）与 WGAN-GP 数学推导（对应 networks.py） 15

3.2.1 从白板一最小二乘（LS）导出 Chinese_Flower 默认的 LSGAN 15

3.2.2 WGAN-GP 与二阶自动微分梯度惩罚（cal_gradient_penalty） 15

3.3 CycleGAN 双向循环建模流程（Chinese_Flower 核心架构推导） 16

3.3.1 数学符号与 models/cycle_gan_model.py 变量名精确对照表 16

3.3.2 生成器总目标函数（对应 cycle_gan_model.py 第 113–138 行） 17

第 4 章 · 数据科学流程 Stage 1–4 源码精讲（爬虫、清洗、变换与缓冲存储） 17

4.1 Stage 1: Data Extraction（数据抽取）——故宫画作爬虫 `crawler.py` 与 `util/get_data.py` . 17

4.1.1 故宫博物院数字文物库爬虫（`crawler.py`）核心机制拆解 17

4.1.2 伯克利数据集爬虫（`util/get_data.py`） 18

4.2 Stage 2 & 3: Data Pre-Processing & Transformation（数据清洗与转换）——`data/` 18

4.3 Stage 4: Data Storage（数据存储与历史缓冲池 `ImagePool`） 18

第 5 章 · 数据科学流程 Stage 5–6 源码精讲（网络架构、训练诊断、微调与推理决策） 19

5.1 生成器 `ResnetGenerator`（9-blocks）张量维度追踪 19

5.2 判别器 `NLayerDiscriminator`：70×70 PatchGAN 感受野数学推导 19

5.3 训练循环（`train.py`）、`train_log` 诊断与微调推理（`finetune/` & `test.py`） 20

附录：课堂白板公式、GAN 公式与项目命令速查表 20

导读：如何用「数据科学标准流程」与「机器学习标准流程」贯通课堂数学、GAN 原理与 `Chinese_Flower` 项目？

本文档旨在帮助你从零开始、彻底读懂 `Chinese_Flower`（基于 CycleGAN 的中式花鸟画风格转换）项目的每一行核心代码与每一步数学推导。全文严格按照课堂讲授的两大标准流程进行组织：

1. 第一主线：数据科学六步标准流程（**Data Science Standard Pipeline**，对应课堂课件 **1.1–1.3**）遵循课堂课件 *1.1 Pipeline*（参考自 *Machine Learning with Big Data: Challenges and Approaches*）的六阶段标准流水线，将 `Chinese_Flower` 项目的工程源码严丝合缝地映射到六个标准阶段：

   $$\text{Data Extraction} \longrightarrow \underbrace{\text{Data Pre-Processing}}_{\text{Data Manipulations}} \longrightarrow \underbrace{\text{Data Transformation}}_{\text{Processing Manipulations}} \longrightarrow \text{Data Storage} \longrightarrow \underbrace{\text{Data Analysis}}_{\text{Algorithm Manipulations}} \longrightarrow \text{Dec}$$

2. 第二主线：机器学习五步标准流程（**Machine Learning Standard Workflow**，对应三张课堂白板板书）无论是课堂白板上的线性回归（**Linear Regression**）、逻辑回归（**Logistic Regression**），还是本项目的 **GAN / LSGAN / WGAN-GP / CycleGAN**，在机器学习建模上都严格遵循以下五步标准范式：

   - **Step 1** —数据集与特征空间定义（**Dataset & Feature Space Definition**）：明确输入特征矩阵X_{input}（行 `samples`、列 `features`）、目标空间y_{GT}（连续值$y \in \mathbb{R}$、二分类标签$y \in \{0, 1\}$ 或高维图像流形$\mathcal{X}$）与概率分布假设；
   - **Step 2** —模型假设与参数化（**Model Parameterization** $f(X; w, c)$）：设计从输入到输出的函数映射结构（线性变换$Xw + c$、Sigmoid概率映射$\sigma(x_i^\top w + c)$，或深度残差生成器G 与 PatchGAN 判别器D）；
   - **Step 3** —损失函数构造与极大似然估计（**Loss Function & MLE Construction**）：从欧氏距离导出最小二乘损失（**Least Squares,** $\min \|\hat{y} - y\|_2^2$），或从伯努利分布导出极大似然估计（**MLE** “**Log it**” $\rightarrow$ 负对数似然 / 二元交叉熵 **BCE**）；
   - **Step 4** —最优化求解与数学推导（**Optimization & Mathematical Derivation**）：通过几何正交投影求解正规方程闭式解$\hat{x} = (A^\top A)^{-1} A^\top y$，或通过变分求导与反向传播梯度下降（SGD / Adam）求解最优参数$(\hat{w}, \hat{c}) = \arg\min \text{Loss}$；
   - **Step 5** —预测与推理决策（**Prediction / Inference**）：冻结预训练模型（`pre-train model`）参数$(\hat{w}, \hat{c})$，输入新样本计算预测输出并与阈值$P_{\text{Threshold}}$ 比较或还原生成图像。

   打通课堂白板与 **`Chinese_Flower`** 的两座核心数学桥梁：- 课堂白板二、三的「逻辑回归 **MLE**」$\Longleftrightarrow$ 原始 **GAN**（**Vanilla GAN**）的判别器（当判别器使用 Sigmoid + 二元交叉熵时，它本质上就是一个区分真图$y = 1$ 与假图$y = 0$ 的逻辑回归分类器 !）；- 课堂白板一的「线性回归最小二乘（**LS**）」$\Longleftrightarrow$ **`Chinese_Flower`** 默认使用的 **LSGAN**（最小二乘 **GAN**）（用二次方损失$\|D(x) - y\|_2^2$ 替代逻辑回归交叉熵，彻底解决 Sigmoid 饱和区导致的梯度消失 !）。

第 1 章 · 数据科学六步标准流程全景（对应课堂课件 1.1–1.3 与项目架构映射）

1.1 数据科学标准流水线（1.1 Pipeline）与 Chinese_Flower 全模块映射

根据课堂课件 *1.1 Pipeline*，一个完整的数据科学与机器学习工程系统由六个顺序阶段及三大操作层级（**Data Manipulations**、**Processing Manipulations**、**Algorithm Manipulations**）构成。下表展示了课堂理论框架与 `Chinese_Flower` 项目源码的精确映射关系：

流水线阶段（1.1 Pipeline）	顶层操作层级	课堂课件核心定义	Chinese_Flower 对应源码文件	项目中的具体实现内容
Stage 1: Data Extraction（数据抽取）	数据获取层	Web scraping、API integration、Text pattern matching、OCR、Data mining、NLP、Database querying	`crawler.pyutil/get_data.py`	1. 调用故宫博物院数字文物库 JSON API（`cateList=17` 绘画类）并流式爬取馆藏中国画；2. 用 `BeautifulSoup` 抓取 UC Berkeley 官方数据集压缩包。
Stage 2: Data Pre-Processing（数据预处理）	**Data Manipulations**（数据层操作）	**Data Cleaning**（清洗无关数据、重复数据、噪声数据、错误类型、缺失值、多重共线性）	`crawler.pydata/image_folder.pydata/unaligned_dataset.py`	1. 哈希去重跳过已存在重复文件；2. `is_image_file()` 过滤非图像扩展名；3. `Image.open().convert('RGB')` 统一将灰度/RGBA 异常格式清洗为标准 3 通道 RGB。
Stage 3: Data Transformation（数据转换与规约）	**Processing Manipulations**（处理层操作）	**Data Transformation**（处理异常值 Outliers、不合规格式 Unacceptable format）与 **Data Reduction**（维度规约、类别规约）	`data/base_dataset.pyfine-tuning-model.pymodels/networks.pysegmentation.pymodels/ne`	1. 双三次插值缩放至286×286并随机裁剪为256×256；2. 像素值从$[0, 255]$线性归一化至$[-1, 1]$；3. 生成器编码器将256×256空间降维规约至64×64潜在特征图。
Stage 4: Data Storage（数据存储与缓冲）	存储与 I/O 层	结构化/非结构化数据集组织、内存缓存与持久化存储	`data/__init__.pyutil/image_pool.pymodels/base_model.py`	1. 按 `trainA`（白描图）/ `trainB`（国画）分目录存储；2. `DataLoader` 启用 `pin_memory=True` 锁页内存与多进程预取；3. `ImagePool(50)` 维护历史生成图像环形缓冲区；4. 每轮保存`.pth` 权重检查点。

流水线阶段（1.1 Pipeline）	顶层操作层级	课堂课件核心定义	Chinese_Flower 对应源码文件	项目中的具体实现内容
Stage 5: Data Analysis（数据分析与算法建模）	**Algorithm Manipulations**（算法层操作）	机器学习模型构建、损失函数设计、参数优化与训练诊断	`models/cycle_gan_model.py`、`models/networks.py`、`train.py`、`train_log`	1. 构建 9-block ResNet 生成器与 70×70 PatchGAN 判别器；2. 联合优化 LSGAN 对抗损失、L_1 循环一致性损失与身份损失；3. 通过 `train_log` 与 TensorBoard 分析 8 项损失收敛曲线。
Stage 6: Decision Making（决策与推理部署）	推理与应用层	使用预训练模型执行预测/生成、阈值判定与结果输出	`test.py`、`models/test_model.py`	1. 仅加载预训练生成器 `netG_A` 并切换至 `.eval()` 模式；2. 在 `torch.no_grad()` 下对512×512 测试图像前向推理；3. 经 `tensor2im` 反归一化输出高清中式花鸟画至 `results/`。

1.2 课堂课件 1.2 & 1.3 核心规范完整梳理

1.2.1 七大常见数据抽取技术（1.2 Data Extraction）

课件 *1.2 Data Extraction*（参考自 *7 common data extraction techniques for efficient information retrieval*）列出了获取原始数据的 7 种标准技术：1. **Web scraping**（网页爬虫）：从网站自动提取数据的自动化流程（对应 `util/get_data.py` 解析 HTML 页面提取下载链接，以及 `crawler.py` 流式下载图像）；2. **API integration**（接口集成）：通过 HTTP 协议直接调用后端接口获取结构化 JSON 数据（对应 `crawler.py` POST 请求故宫 `https://digicol.dpm.org.cn/cultural/queryList` 接口）；3. **Text pattern matching**（文本模式匹配）：利用正则表达式或字符串规则解析结构化字段（对应 `crawler.py` 解析原始请求头字符串）；4. **Optical character recognition (OCR)**：光学字符识别；5. **Data mining**（数据挖掘）、**Natural language processing (NLP)** 与 **Database querying**（数据库查询）。

1.2.2 数据预处理三大分支与问题解决方案（1.3 Data Pre-Processing）

课件 *1.3 Data Pre-Processing* 将数据预处理划分为 **Data Cleaning**（数据清洗）、**Data Transformation**（数据转换）与 **Data Reduction**（数据规约）三大支柱，并给出了标准处置规范：

预处理分支	核心问题（Issue）	课堂课件标准解决方案（Standard Solution）	在 Chinese_Flower 项目中的工程实现
Data Cleaning（数据清洗）	**1. Irrelevant data**（无关数据）	检查后直接丢弃（Discard after inspection）；必要时咨询领域专家判断属性相关性	在故宫接口中指定 `cateList="17"`（绘画类），剔除青铜器、瓷器等无关类别
	2. Duplicate data（重复数据）	删除重复行列；若出现数据冲突，回溯原始数据源进行集成对齐	`download_file` 中检查 `if os.path.exists(file_path) and check_exists: return True`
	3. Noisy data（噪声数据）	通过检查识别噪声类型，拒绝无效与越界数据，标准化不一致的类别项	过滤无法打开的损坏图像；在添加高斯噪声增强后使用 `torch.clamp(img_tensor, 0, 1)` 截断越界值
	4. Incorrect data type（数据类型错误）	强制修正数据类型与通道格式	调用 `Image.open(path).convert('RGB')` 将单通道灰度或 4 通道 RGBA 统一转为 3 通道 RGB，并由 `ToTensor()` 转为 `float32`
	5. Missing values（缺失值）	删除缺失记录或使用插值算法填补	图像缩放与边界外推时使用双三次插值（`Image.BICUBIC`）与反射填充（`ReflectionPad2d`）
	6. Multi-collinearity（多重共线性）	消除高度共线的冗余特征维度	通过 `InstanceNorm2d` 对每张图像各通道独立去均值、除标准差，消除光照与对比度共线性
Data Transformation（数据转换）	**Outliers & Unacceptable format**（异常值与格式不合规）	统一空间尺寸与数值量纲区间	`Resize(286)` → `RandomCrop(256)` 统一分辨率；`Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))` 将$[0,255]$ 统一映射到$[-1,1]$
Data Reduction（数据规约）	**Too many dimensions / categories**（维度过高/类别过多）	潜在空间特征降维或局部窗口分解	1. 生成器通过 2 层步幅卷积将256×256 降维至64×64 潜在空间；2. PatchGAN 判别器将全图判别规约为70×70 局部感受野判别

第 2 章 · 机器学习五步标准流程与数学基础（课堂三张白板完整还原）

在进入 GAN 之前，我们先完整还原课堂三张白板上的推导。三张白板不仅清晰展示了机器学习五步标准流程（**Dataset** → **Model** f → **Loss / MLE** → **Optimization** $\arg\min$ → **Prediction / Inference**），而且直接给出了 `Chinese_Flower` 中 **LSGAN**（源自白板一的最小二乘）与 **Vanilla GAN**（源自白板二、三的逻辑回归 MLE）的数学母体。

2.1 白板一精讲：线性变换、线性回归与最小二乘（LS）正交投影推导

Step 1：数据集与线性变换空间定义（Dataset & Linear Transformation）

观察第一张白板左侧与右上角：- 线性变换（**Linear Transformation**）：矩阵$A_{k\times n}$ 定义了一个从n 维向量空间$\mathbb{R}^n$ 到k 维向量空间$\mathbb{R}^k$ 的线性映射：

$$x_n \in \mathbb{R}^n \xrightarrow{A_{k\times n}} y_k = Ax_n \in \mathbb{R}^k$$

- 回归数据集定义（**Regression Dataset**）：给定带有真值标签（Ground Truth, GT）的数据集Dataset：$(X_{\text{input}}, y)_{\text{GT}}$：- 输入特征矩阵$X \in \mathbb{R}^{k\times n}$：每一行（**row**）代表一个样本（samples: $1, 2, \ldots, k$），每一列（**column**）代表一个特征（features: $x_1, x_2, x_3, \ldots, x_n$）；- 输出连续目标向量：$y = (y_1, y_2, \ldots, y_k)^\top \in \mathbb{R}^k$（单样本满足$X_{\text{input}} \in \mathbb{R}^n \xrightarrow{f} y \in \mathbb{R}$）。

Step 2：线性回归模型参数化（Model Parameterization）

对于单个样本$(\vec{x}_i, y_i)$（对应白板二上半部分的 `one sample`），模型假设预测值是特征的线性加权和加上偏置$c \in \mathbb{R}$（白板一写作$c = w_0 \in \mathbb{R}$）：

$$f(\vec{x}_i) = \sum_{j=1}^{n} w_j x_{ij} + c = \vec{x}_i^\top w + c$$

写成全体k 个样本的矩阵形式（若将偏置c 合并进增广设计矩阵$A = [X, \mathbf{1}] \in \mathbb{R}^{k\times(n+1)}$，待求参数向量记为$x$ 或w）：

$$f(X) = Xw + c = \hat{y} \in \mathbb{R}^k$$

Step 3：超定方程组困境与二范数平方损失构造（Loss Construction）

白板一右侧提出了一个关键问题（标注为 `Q:`）：> 在机器学习中，样本数量k 往往远大于特征数量n（即$k \gg n$）。此时线性方程组 >

$$A_{k\times n}x_{n\times 1} = y_{k\times 1} \implies x = ?$$

> 是一个超定方程组（**Overdetermined System**），通常不存在精确解！从几何上看，当参数向量x 取遍$\mathbb{R}^n$ 时，预测向量Ax 只能落在$\mathbb{R}^k$ 中的一个n 维子空间平面——即矩阵A 的列空间$\text{Range}(A) = \{Ax \mid x \in \mathbb{R}^n\}$ 上；而受噪声影响的真实观测向量$y \in \mathbb{R}^k$ 几乎必然落在该平面之外。

为此，机器学习标准流程定义预测向量$\hat{y} = Ax$ 与真值向量y 之间的欧氏距离平方（**Squared L_2 Norm**）作为损失函数：

$$\text{Loss / Optimization:} \quad \min_{w,c} \|\hat{y} - y\|_2^2 \quad \Longleftrightarrow \quad \min_{x \in \mathbb{R}^n} \|Ax - y\|_2^2$$

Step 4：最小二乘（Least Squares, LS）几何正交投影与正规方程推导

观察白板一左下角的子空间投影几何图与底部推导公式：

1. 正交投影几何原理：在$\mathbb{R}^k$ 中，要找列空间平面Range(A) 上距离外部点y 最近的点$A\hat{x}$，由垂线段最短原理（勾股定理），连接最佳预测点$A\hat{x}$ 与真实点y 的残差向量（**Residual Vector**）$y - A\hat{x}$ 必须与平面Range(A) 垂直（正交）！即对任意$x \in \mathbb{R}^n$，残差向量都与Ax 正交：

$$(y - A\hat{x}) \perp Ax, \qquad \forall x \in \mathbb{R}^n$$

2. 正规方程（**Normal Equation**）严格代数推导：两个向量正交等价于它们的内积为0：

$$(Ax)^\top (y - A\hat{x}) = 0, \qquad \forall x \in \mathbb{R}^n$$

利用矩阵转置法则$(Ax)^\top = x^\top A^\top$ 及乘法分配律展开：

$$x^\top A^\top (y - A\hat{x}) = 0 \implies x^\top \left(A^\top y - A^\top A\hat{x}\right) = 0, \qquad \forall x \in \mathbb{R}^n$$

因为该内积对$\mathbb{R}^n$ 中的任意向量x 都恒等于0，括号内的向量必然是零向量：

$$A^\top y - A^\top A\hat{x} = \mathbf{0} \implies (A^\top A)\hat{x} = A^\top y$$

当特征列线性无关（rank$(A) = n$，此时格拉姆矩阵$A^\top A \in \mathbb{R}^{n \times n}$ 对称正定可逆）时，左乘$(A^\top A)^{-1}$ 即得到白板右下角的最小二乘闭式解析解：

$$\hat{x} = (A^\top A)^{-1}(A^\top y)$$

2.2 白板二与白板三精讲：逻辑回归（Logistic Regression）与极大似然估计（MLE）

当机器学习任务从连续值回归转向二元分类（**Binary Classification**，如判别器判断图像是“真国画1”还是“生成假画0”）时，我们进入白板二与白板三的标准推导流程。

Step 1：二分类数据集定义（Binary Classification Dataset）

观察白板二下半部分：- 数据集：Dataset : $(\vec{x}_i, y_i)_{i=1}^n$，输入特征矩阵$X_{n\times k}$ 通过模型f 映射到二元集合：

$$X_{n\times k} \xrightarrow{\text{model } f} y \in \{0, 1\}$$

其中0 代表负类（negative），1 代表正类（positive）。

Step 2：Sigmoid 激活函数与后验概率参数化（Model Parameterization）

为了将实数域上的线性打分$z_i = x_i^\top w + c \in \mathbb{R}$ 转化为合法的概率值$P \in (0, 1)$，引入 **Sigmoid** 函数（白板二左下角曲线图）：

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1)$$

由此将正类与负类的条件概率建模为（白板二右下角两个方框）：

$$P(y = 1 \mid X = x_i) = \sigma(x_i^\top w + c) = \frac{1}{1 + e^{-(x_i^\top w + c)}}$$
$$P(y = 0 \mid X = x_i) = 1 - \sigma(x_i^\top w + c) = \frac{1}{1 + e^{+(x_i^\top w + c)}}$$

Step 3：伯努利分布合成与极大似然估计（MLE “Log it”）

1. 单样本概率统一表达式（白板二红框公式）：对于任意样本$(\vec{x}_i, y_i)$（其中$y_i \in \{0, 1\}$），上述分段公式可统一写为伯努利分布概率质量函数：

   $$P(\vec{x}_i, y_i) = \sigma^{y_i}(1 - \sigma)^{1-y_i}$$

   （当$y_i = 1$ 时等于σ；当$y_i = 0$ 时等于$1 - \sigma$）。
2. 极大似然估计（**MLE**）与**"Log it"** 对数转换（白板二底部核心框）：由样本独立同分布假设，联合似然函数为$\prod_{i=1}^n P(\vec{x}_i, y_i)$。对其取自然对数（白板标注**"Log it"**），将连乘转化为连加：

$$\max_{w,c} \prod_{i=1}^n P(\vec{x}_i, y_i) \xrightarrow{\text{"Log it"}} \max_{w,c} \sum_{i=1}^n \log\left(\sigma^{y_i} \cdot (1 - \sigma)^{1-y_i}\right) = \max_{w,c} \sum_{i=1}^n \Big[y_i \log \sigma + (1 - y_i) \log(1 - \sigma)\Big]$$

Step 4：白板三的负对数似然最小化（Minimization）与$\{-1, +1\}$ 标签统一形式

1. 导出白板三顶部的最小化损失公式（对应 PyTorch `BCEWithLogitsLoss`）　观察白板三顶部：最大化对数似然等价于最小化负对数似然（`minimization: Loss ↓`）。记线性项$z_i = x_i^\top w + c$，利用$\sigma = \frac{1}{1+e^{-z_i}} = \frac{e^{z_i}}{1+e^{z_i}}$与$1 - \sigma = \frac{1}{1+e^{z_i}}$，有对数几率$\log \frac{\sigma}{1-\sigma} = z_i = x_i^\top w + c$。将单样本负对数似然化简：

$$
\begin{aligned}
-\log P(\vec{x}_i, y_i) &= -y_i \log \sigma - (1-y_i)\log(1-\sigma) \\
&= -y_i \log\left(\frac{\sigma}{1-\sigma}\right) - \log(1-\sigma) \\
&= -y_i(x_i^\top w + c) - \log\left(\frac{1}{1+e^{x_i^\top w + c}}\right) \\
&= -y_i(x_i^\top w + c) + \log\left(1+e^{x_i^\top w + c}\right)
\end{aligned}
$$

这就一字不差地得到了白板三顶部的最优化目标：

$$
(\hat{w}, \hat{c}) = \arg\min_{w,c} \sum_{i=1}^{n} \left\{ -y_i(x_i^\top w + c) + \log\left(1+e^{x_i^\top w + c}\right) \right\}
$$

代码映射：在 `Chinese_Flower` 的 `models/networks.py` 第 116 行，当设置 `gan_mode == 'vanilla'` 时，代码调用的 `nn.BCEWithLogitsLoss()` 正是直接计算这个化简后的公式！

2. 导出白板三底部红字的$\{-1,+1\}$ 标签对称公式 观察白板三底部红字 `Label: (0, 1) -> (-1, 1)`：若将正负类标签编码从$\{0,1\}$ 改为对称的$y_i \in \{-1,+1\}$，利用$1-\sigma(z_i) = \frac{1}{1+e^{+z_i}} = \sigma(-z_i)$：

$$
\begin{cases}
P(y_i = +1 \mid x = x_i) = \sigma = \dfrac{1}{1+e^{-(x_i^\top w + c)}} \\
P(y_i = -1 \mid x = x_i) = 1-\sigma = \dfrac{1}{1+e^{+(x_i^\top w + c)}}
\end{cases}
\Longrightarrow P(x_i, y_i) = \frac{1}{1+e^{-y_i(x_i^\top w + c)}}
$$

Step 5：基于预训练模型的预测与阈值推理（Prediction / Inference）

观察白板三中部：- 训练收敛得到最优参数$(\hat{w}, \hat{c})$ 后，模型成为固化的预训练模型（**`pre-train model`**）；- 推理阶段（`Prediction: / Inference:`），对输入样本$\vec{x}_i$ 计算预测概率$p = \sigma(\vec{x}_i^\top \hat{w} + \hat{c}) \in (0,1)$，并与阈值$P_{\text{Threshold}}$（通常为0.5）比较输出分类决策：

$$
\text{Output} = \begin{cases} 1, & p > P_{\text{Threshold}} \\ 0, & p < P_{\text{Threshold}} \end{cases}
$$

2.3 生成模型必备的信息论与深度学习数学工具

在把白板上的逻辑回归与最小二乘推广为 GAN 之前，我们还需要补充三项核心数学工具：

2.3.1 KL 散度、吉布斯不等式证明与 JS 散度

1. **KL** 散度（相对熵）与极大似然估计（**MLE**）的等价性：对两个连续概率密度$p(x)$和$q(x)$，定义 KL 散度为：

$$\mathrm{KL}(p \,\|\, q) = \int_{\mathcal{X}} p(x) \log \frac{p(x)}{q(x)}\, dx = \mathbb{E}_{x\sim p(x)}[\log p(x)] - \mathbb{E}_{x\sim p(x)}[\log q(x)]$$

   在显式生成模型中，最大化对数似然$\max_\theta \mathbb{E}_{x\sim p_{\text{data}}}[\log p_\theta(x)]$ 在数学上严格等价于最小化前向 **KL** 散度$\min_\theta \mathrm{KL}(p_{\textbf{data}} \,\|\, p_\theta)$。

2. 定理 **2.1**（**KL** 散度非负性证明）：因为$\varphi(t) = -\log t$ 的二阶导数$\varphi''(t) = \frac{1}{t^2} > 0$（严格凸函数），由琴生不等式（Jensen's Inequality）$\mathbb{E}[\varphi(T)] \geq \varphi(\mathbb{E}[T])$，取$T = \frac{q(x)}{p(x)}$：

$$\mathrm{KL}(p \,\|\, q) = \mathbb{E}_{x\sim p}\left[-\log \frac{q(x)}{p(x)}\right] \geq -\log \mathbb{E}_{x\sim p}\left[\frac{q(x)}{p(x)}\right] = -\log \int_{\mathcal{X}} q(x)\, dx = -\log 1 = 0$$

   当且仅当$p(x) = q(x)$ 几乎处处成立时取等号。

3. 前向 **KL** 与反向 **KL** 的非对称性（解释模糊与模式崩溃）：

   - 前向 **KL** $\mathrm{KL}(p_{\textbf{data}} \,\|\, p_g)$（零回避 / **Mode-Covering**）：凡是$p_{\text{data}}(x) > 0$ 处若$p_g(x) \to 0$，则代价$\to +\infty$，迫使模型覆盖所有模式但易在多峰间产生模糊均值；
   - 反向 **KL** $\mathrm{KL}(p_g \,\|\, p_{\textbf{data}})$（寻峰 / **Mode-Seeking**）：凡是$p_{\text{data}}(x) = 0$ 处若$p_g(x) > 0$，则代价$\to +\infty$；但若直接放弃某些真实峰（令$p_g(x) = 0$），代价为$0 \log 0 = 0$。这导致生成器倾向于只坍缩在少数真实峰值上，引发模式崩溃（**Mode Collapse**）。

4. **JS** 散度（**Jensen-Shannon Divergence**）：

$$\mathrm{JSD}(p \,\|\, q) = \frac{1}{2}\mathrm{KL}\left(p \,\middle\|\, \frac{p+q}{2}\right) + \frac{1}{2}\mathrm{KL}\left(q \,\middle\|\, \frac{p+q}{2}\right) \in [0, \log 2]$$

2.3.2 对抗训练中的复合函数链式法则与 Adam 优化器

1. 为什么更新生成器G 时冻结判别器参数（`set_requires_grad(D, False)`），梯度仍能传给G？由复合函数雅可比链式法则，生成器损失$\mathcal{L}_G = \ell(D(G(z;\theta_G));\theta_D)$ 对生成器参数θ_G 的梯度为：

$$\frac{\partial \mathcal{L}_G}{\partial \theta_G} = \frac{\partial \ell}{\partial D} \cdot \underbrace{\left.\frac{\partial D(\tilde{x};\theta_D)}{\partial \tilde{x}}\right|_{\tilde{x}=G(z)}}_{\text{判别器对输入图像的导数}} \cdot \frac{\partial G(z;\theta_G)}{\partial \theta_G}$$

   关闭判别器权重的 `requires_grad` 只是省去了计算判别器自身参数梯度$\frac{\partial \mathcal{L}_G}{\partial \theta_D}$ 的开销，而误差关于输入图像$\tilde{x}$ 的导数$\frac{\partial D(\tilde{x})}{\partial \tilde{x}}$ 依然完整计算并回传给生成器！反之，在更新判别器D 时调用 `fake.detach()` 则是直接切断$\tilde{x}$ 与生成器的计算图连接，防止回传到G。

2. **Adam** 优化器公式与$\beta_1 = 0.5$ 设置：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

`Chinese_Flower` 将β_1 设为0.5（而非默认的0.9），是为了减小极小极大鞍点博弈中的历史动量惯性，抑制生成器和判别器围绕均衡点发生旋转振荡。

2.3.3 卷积、转置卷积尺寸公式与 `InstanceNorm2d` 数学原理

1. 卷积（**`Conv2d`**）与转置卷积（**`ConvTranspose2d`**）输出空间尺寸公式：

$$H_{\text{out}}^{\text{conv}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1, \qquad H_{\text{out}}^{\text{deconv}} = (H_{\text{in}} - 1) \times s - 2p + k + \text{output_padding}$$

2. 为什么风格迁移使用 **`InstanceNorm2d`** 而非 **`BatchNorm2d`**？设特征图$x \in \mathbb{R}^{N \times C \times H \times W}$。`InstanceNorm2d` 仅在单张图像$n$、单通道$c$ 的空间平面(H, W) 上独立计算均值与方差：

$$\mu_{nc} = \frac{1}{HW} \sum_{h=1}^{H} \sum_{w=1}^{W} x_{nchw}, \qquad \sigma_{nc}^2 = \frac{1}{HW} \sum_{h=1}^{H} \sum_{w=1}^{W} (x_{nchw} - \mu_{nc})^2, \qquad \hat{x}_{nchw} = \frac{x_{nchw} - \mu_{nc}}{\sqrt{\sigma_{nc}^2 + \epsilon}}$$

这能够直接剥离单张输入照片的个体光照和对比度偏差，同时避免同一 Batch 内不同图像的色彩统计量互相干扰。

第 3 章 · 按「机器学习五步标准流程」推导 GAN 家族与 CycleGAN 原理

本章严格按照课堂白板的机器学习五步标准建模流程，从白板上的逻辑回归与最小二乘一步步推导出 **Vanilla GAN**、**LSGAN**、**WGAN-GP** 与 **CycleGAN**。

3.1 原始 GAN（Vanilla GAN）五步标准建模与变分推导

Step 1 & Step 2：分布定义与双网络参数化

- 真实样本：$x \sim p_{\text{data}}(x)$，对应白板二逻辑回归中的正类标签$y = 1$；
- 生成样本：隐变量$z \sim p_z(z)$ 经生成器映射为$\tilde{x} = G(z; \theta_g) \sim p_g(x)$，对应白板二逻辑回归中的负类标签$y = 0$；
- 判别器：$D(x; \theta_d) = \sigma(f_D(x; \theta_d)) \in (0, 1)$ 预测输入x 来自真实数据分布的后验概率$P(y = 1 \mid x)$。

Step 3：直接由白板二逻辑回归 MLE 导出极小极大博弈目标$V(D,G)$

将白板二底部的逻辑回归对数似然$\sum[y_i \log \sigma(x_i) + (1-y_i)\log(1-\sigma(x_i))]$ 按正类（$y_i=1, x_i \sim p_{\text{data}}$）和负类（$y_i=0, x_i=G(z)\sim p_g$）两部分求期望，立即得到判别器要最大化、生成器要最小化的价值函数：

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1-D(G(z)))]$$

Step 4：变分极值推导（证明最优判别器D_G^* 与 JS 散度等价性）

1. 测度变换（**LOTUS**）：将第二项对$z\sim p_z(z)$ 的积分改写为对$x\sim p_g(x)$ 的积分：

$$V(D,G) = \int_{\mathcal{X}} \Big[p_{\text{data}}(x)\log D(x) + p_g(x)\log(1-D(x))\Big]dx$$

2. 固定生成器G，逐点求最优判别器$D_G^*(x)$：记$a=p_{\text{data}}(x), b=p_g(x), y=D(x)\in(0,1)$，对标量被积函数$h(y)=a\log y + b\log(1-y)$ 求导：

$$\frac{dh(y)}{dy} = \frac{a}{y} - \frac{b}{1-y} = 0 \implies y^* = \frac{a}{a+b}, \qquad \frac{d^2h(y)}{dy^2} = -\frac{a}{y^2} - \frac{b}{(1-y)^2} < 0$$

故最优判别器闭式解为：

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_g(x)}$$

3. 代入$D_G^*(x)$ 求解最优生成器G^*：将$D_G^*(x)$ 代回$V(D,G)$，令混合分布$M(x)=\frac{p_{\text{data}}(x)+p_g(x)}{2}$：

$$\begin{aligned} C(G) = \max_D V(D,G) &= \int_{\mathcal{X}} \left[p_{\text{data}}(x)\log\frac{p_{\text{data}}(x)}{2M(x)} + p_g(x)\log\frac{p_g(x)}{2M(x)}\right]dx \\ &= -\log 4 + \text{KL}(p_{\text{data}} \,\|\, M) + \text{KL}(p_g \,\|\, M) \\ &= -\log 4 + 2\cdot \text{JSD}(p_{\text{data}} \,\|\, p_g) \end{aligned}$$

当且仅当$p_g(x)=p_{\text{data}}(x)$ 时，$\text{JSD}(p_{\text{data}}\,\|\,p_g)=0$，达到全局最小值$C(G^*)=-\log 4$。

Step 5：纳什均衡与白板三决策阈值$P_{\text{Threshold}}=0.5$ 的印证

当达到全局最优$p_g=p_{\text{data}}$ 时，最优判别器对任意样本输出：

$$D_{G^*}^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_{\text{data}}(x)} = \frac{1}{2} = 0.5 = P_{\text{Threshold}}$$

这与白板三 Step 5 的二分类决策阈值$P_{\text{Threshold}}=0.5$ 完美呼应：此时判别器输出恒等于阈值边界0.5，完全无法区分真画与假画！

3.2 最小二乘 GAN（LSGAN）与 WGAN-GP 数学推导（对应 `networks.py`）

3.2.1 从白板一最小二乘（LS）导出 `Chinese_Flower` 默认的 LSGAN

在原始 GAN 中，当高维空间里的真实画作流形与生成流形不重叠时，$\mathrm{JSD}(p_{\text{data}} \,\|\, p_g) = \log 2$ 为常数，判别器 Sigmoid 进入饱和区（$D(G(z)) \to 0$），导致生成器梯度消失。

`Chinese_Flower` 在 `models/cycle_gan_model.py` 第 48 行默认采用 `self.gan_mode = 'lsgan'`（在 `models/networks.py` 中对应 `nn.MSELoss()`），即直接用白板一的最小二乘损失$\|\hat{y} - y\|_2^2$ 替代白板二的逻辑回归交叉熵：

$$\min_D V_{\text{LSGAN}}(D) = \frac{1}{2}\mathbb{E}_{x\sim p_{\text{data}}}\left[(D(x)-b)^2\right] + \frac{1}{2}\mathbb{E}_{z\sim p_z}\left[(D(G(z))-a)^2\right]$$
$$\min_G V_{\text{LSGAN}}(G) = \frac{1}{2}\mathbb{E}_{z\sim p_z}\left[(D(G(z))-c)^2\right]$$

（项目中默认取 `target_real_label=1.0`, `target_fake_label=0.0`，即$a=0, b=c=1$）。1. 固定G 求最优判别器$D^*(x)$：对被积函数$\frac{1}{2}p_{\text{data}}(x)(D(x)-b)^2 + \frac{1}{2}p_g(x)(D(x)-a)^2$ 关于$D(x)$ 求导并令其为0，得：

$$D^*(x) = \frac{b\,p_{\text{data}}(x) + a\,p_g(x)}{p_{\text{data}}(x) + p_g(x)}$$

2. 代入$D^*(x)$ 导出皮尔逊χ^2 散度：在生成器目标中加上不依赖G 的常数项$\frac{1}{2}\mathbb{E}_{x\sim p_{\text{data}}}[(D(x)-c)^2]$，当编码满足$b-c=1, b-a=2$（或对$a=0, b=c=1$ 乘以缩放因子）时：

$$2C_{\text{LSGAN}}(G) = \int_{\mathcal{X}} \frac{\big(2p_g(x) - (p_{\text{data}}(x)+p_g(x))\big)^2}{p_{\text{data}}(x)+p_g(x)}\,dx = \chi^2_{\text{Pearson}}\big(p_{\text{data}}+p_g \,\|\, 2p_g\big)$$

二次损失$(D(G(z))-1)^2$ 的导数$2(D(G(z))-1)$ 与偏差距离呈线性正比，永不饱和，能持续将偏离流形的生成样本拉向真实花鸟画分布中心。

3.2.2 WGAN-GP 与二阶自动微分梯度惩罚（`cal_gradient_penalty`）

`models/networks.py` 第 151–183 行还实现了 WGAN-GP 的梯度惩罚函数 `cal_gradient_penalty`。基于 Kantorovich-Rubinstein 对偶定理，Wasserstein-1 距离要求判别器满足1-Lipschitz 连续约束（$\|\nabla_x D(x)\|_2 \le 1$）。代码在真实图$x_{\text{real}}$ 与假图x_{fake} 之间随机采样插值点$\hat{x} = \alpha x_{\text{real}} + (1-\alpha)x_{\text{fake}}$，通过 `torch.autograd.grad(..., create_graph=True)` 计算判别器对输入图像的梯度并施加二次惩罚：

$$\mathcal{L}_{\text{GP}} = \lambda_{\text{gp}}\,\mathbb{E}_{\hat{x}}\Big[\big(\big\|\nabla_{\hat{x}} D(\hat{x}) + 10^{-16}\big\|_2 - 1\big)^2\Big]$$

3.3 CycleGAN 双向循环建模流程（Chinese_Flower 核心架构推导）

在无配对数据集中，仅有单向对抗损失会导致生成器把所有风景照都映射成同一幅国画。CycleGAN 通过引入双向对偶生成器(G_A, G_B)、双判别器(D_A, D_B)、循环一致性损失与身份映射损失构成完整闭环。

3.3.1 数学符号与 models/cycle_gan_model.py 变量名精确对照表

阅读 `models/cycle_gan_model.py` 时，最关键的是分清 `netD_A` 与 `netD_B` 各自判别的域：

数学符号	代码属性名	输入张量	输出张量	严格数学与物理定义
$G_A : \mathcal{A} \to \mathcal{B}$	`self.netG_A`	`real_A` (a)	`fake_B` $(\tilde{b} = G_A(a))$	正向生成器：将自然风景照片a 转换为中式花鸟画$\tilde{b}$
$G_B : \mathcal{B} \to \mathcal{A}$	`self.netG_B`	`real_B` (b)	`fake_A` $(\tilde{a} = G_B(b))$	反向生成器：将中式花鸟画b 转换回自然风景照片$\tilde{a}$
$D_A : \mathcal{B} \to \mathbb{R}^{30\times30}$	**`self.netD_A`**	`real_B` 或 `fake_B`	30×30 局部真假评分矩阵	国画域$\mathcal{B}$ 判别器：判断输入图像是真实故宫画作 `real_B` 还是 `netG_A` 伪造的 `fake_B`
$D_B : \mathcal{A} \to \mathbb{R}^{30\times30}$	**`self.netD_B`**	`real_A` 或 `fake_A`	30×30 局部真假评分矩阵	照片域$\mathcal{A}$ 判别器：判断输入图像是真实自然照片 `real_A` 还是 `netG_B` 伪造的 `fake_A`
$\hat{a} = G_B(G_A(a))$	`self.rec_A`	`fake_B`	`rec_A`	正向循环重构图像：自然照片$\xrightarrow{G_A}$ 伪国画$\xrightarrow{G_B}$ 重构照片
$\hat{b} = G_A(G_B(b))$	`self.rec_B`	`fake_A`	`rec_B`	反向循环重构图像：真实国画$\xrightarrow{G_B}$ 伪照片$\xrightarrow{G_A}$ 重构国画
$G_A(b)$	`self.idt_A`	`real_B` (b)	`idt_A`	国画域身份映射：把已经是国画的 `real_B` 输入给转国画网络 `netG_A`，要求保持原样
$G_B(a)$	`self.idt_B`	`real_A` (a)	`idt_B`	照片域身份映射：把已经是照片的 `real_A` 输入给转照片网络 `netG_B`，要求保持原样

3.3.2 生成器总目标函数（对应 `cycle_gan_model.py` 第 113–138 行）

$$\begin{aligned}\mathcal{L}_G = &\underbrace{\mathbb{E}_a\left[\|D_A(G_A(a))-1\|_2^2\right]}_{\texttt{loss_G_A}} + \underbrace{\mathbb{E}_b\left[\|D_B(G_B(b))-1\|_2^2\right]}_{\texttt{loss_G_B}} \\ &+ \underbrace{\lambda_A \mathbb{E}_a\left[\|G_B(G_A(a))-a\|_1\right]}_{\texttt{loss_cycle_A}\ (\lambda_A=10.0)} + \underbrace{\lambda_B \mathbb{E}_b\left[\|G_A(G_B(b))-b\|_1\right]}_{\texttt{loss_cycle_B}\ (\lambda_B=10.0)} \\ &+ \underbrace{\lambda_B \lambda_{\text{idt}} \mathbb{E}_b\left[\|G_A(b)-b\|_1\right]}_{\texttt{loss_idt_A}\ (10.0\times0.5=5.0)} + \underbrace{\lambda_A \lambda_{\text{idt}} \mathbb{E}_a\left[\|G_B(a)-a\|_1\right]}_{\texttt{loss_idt_B}\ (10.0\times0.5=5.0)}\end{aligned}$$

第 4 章 · 数据科学流程 Stage 1–4 源码精讲（爬虫、清洗、变换与缓冲存储）

4.1 Stage 1: Data Extraction（数据抽取）——故宫画作爬虫 `crawler.py` 与 `util/get_data.py`

4.1.1 故宫博物院数字文物库爬虫（`crawler.py`）核心机制拆解

1. 原始请求头字符串解析（**Text Pattern Matching**）：

   ```
   self.headers = dict(
       [[y.strip() for y in x.strip().split(':', 1)]
        for x in self.headers_str.strip().split('\n') if x.strip()]
   )
   ```

   使用 `split(':', 1)` 限制只按每行第一个冒号切分，防止把 `Origin: https://digicol.dpm.org.cn` 中的协议冒号切碎。

2. **POST JSON** 接口集成与动态分页算法（**API Integration**）：`get_one_page` 向 `https://digicol.dpm.org.cn/cultural/quer` 发送表单请求（`cateList="17"` 代表故宫文物中的“绘画”分类）。当第 1 页返回后，读取总文物数 `recordcount`，通过向上取整公式自动修正总页数：

   $$\text{end_page} = \left\lceil \frac{\text{recordcount}}{\text{page_size}} \right\rceil = \left\lceil \frac{\text{recordcount}}{20} \right\rceil$$

   随后将 `self.reset_end_page = False`，每页抓取后执行 `time.sleep(3)` 延时防封禁。

3. 流式分块下载 **`download_file`**：使用 `requests.get(..., stream=True)` 配合 `response.iter_content(chunk_size=10` 每次仅从网络流读取1 KB 写入磁盘，内存占用恒为$O(1)$，并利用 `\r` 实时刷新下载进度百分比$\frac{\text{size}}{\text{content_size}} \times 100\%$。

   - 跨平台提示：第 127 行 `save_name = fr'{self.folder}\images\{self.spider_num}_{name}.png'` 使用了 Windows 反斜杠，在 macOS/Linux 下建议改为 `os.path.join(self.folder, 'images', f'{self.spider_num}_{name}.png')`。

4.1.2 伯克利数据集爬虫（`util/get_data.py`）

使用 `BeautifulSoup(r.text, 'lxml')` 解析 HTML 目录页中的所有 `<a href="...">` 标签，筛选`.zip`或`.tar.gz`数据集包下载并调用 `ZipFile` / `tarfile` 自动解压。

4.2 Stage 2 & 3: Data Pre-Processing & Transformation（数据清洗与转换）——`data/`

1. **Data Cleaning**（清洗重复、非图像与异常通道）：
   - `download_file` 通过 `os.path.exists(file_path)` 避免重复下载；
   - `data/image_folder.py` 通过 `is_image_file()` 过滤非图像文件；
   - `data/unaligned_dataset.py` 第 57–58 行调用 `Image.open(path).convert('RGB')`，将可能混入的单通道灰度图或 4 通道 RGBA 图强制清洗为标准 3 通道 RGB。
2. **Data Transformation**（空间裁剪与$[-1,1]$ 对称归一化闭环）：
   - 空间变换：`transforms.Resize([286, 286], Image.BICUBIC)` → `transforms.RandomCrop(256)`；
   - 正向归一化（**`base_dataset.py`**）：

$$x_{\text{in}} = \frac{I/255.0 - 0.5}{0.5} = 2 \cdot \frac{I}{255.0} - 1.0 \in [-1.0, +1.0]$$

     与生成器末端 `nn.Tanh()` 的值域$(-1,1)$ 严格对齐；
   - 逆向还原（**`util/util.py`** 中的 **`tensor2im`**）：

$$I_{\text{out}} = \text{uint8}\left(\frac{x_{\text{out}} + 1.0}{2.0} \times 255.0\right) \in [0, 255]$$

3. 无配对随机采样打破伪关联（**`UnalignedDataset.__getitem__`**）：当 `serial_batches=False` 时，域A 使用确定性索引 `index % self.A_size`，而域B 使用 `random.randint(0, self.B_size - 1)` 随机采样，确保每次迭代送入网络的(a, b) 都是独立抽取的无配对样本对。

4.3 Stage 4: Data Storage（数据存储与历史缓冲池 `ImagePool`）

1. 多进程异步预取与锁页内存(**`data/__init__.py`**):`DataLoader` 配置 `pin_memory=True` 与 `persistent_workers=True`，加速 CPU 内存向 GPU 显存的 DMA 传输。
2. 历史图像缓冲池 **`ImagePool`**（**`util/image_pool.py`**）：维护容量为 `pool_size = 50` 的历史假图列表。当池满后，每次 `query()` 以50% 概率直接返回当前生成的假图，以另外50% 概率随机取出一张历史假图送给判别器训练并将当前假图存入池中。这使判别器面对的是过去一段时间的平均生成分布，防止生成器和判别器陷入周期性“猫鼠追逐”振荡。

第 5 章 · 数据科学流程 Stage 5–6 源码精讲（网络架构、训练诊断、微调与推理决策）

5.1 生成器 ResnetGenerator（9-blocks）张量维度追踪

`models/networks.py` 第 186–269 行的 `ResnetGenerator` 以$(B, 3, 256, 256)$ 为输入，逐层变换如下：

阶段	层结构（`models/networks.py`）	卷积参数(k, s, p, op)	输出张量维度	核心作用
首层特征提取	`ReflectionPad2d(3)` + `Conv2d(3, 64)` + `IN` + `ReLU`	$k=7, s=1, p=3$(反射)	$(B, 64, 256, 256)$	反射填充消除画幅边缘黑框伪影
下采样层 1	`Conv2d(64, 128)` + `IN` + `ReLU`	$k=3, s=2, p=1$	$(B, 128, 128, 128)$	空间减半，通道翻倍（Data Reduction）
下采样层 2	`Conv2d(128, 256)` + `IN` + `ReLU`	$k=3, s=2, p=1$	$(B, 256, 64, 64)$	规约至64×64 紧凑特征空间
9 个残差块	$9\times$ `ResnetBlock(256)`: `x + conv_block(x)`	$k=3, s=1, p=1$(反射)	$(B, 256, 64, 64)$	恒等支路保留原图几何构图，残差支路学习水墨笔触
上采样层 1	`ConvTranspose2d(256, 128)` + `IN` + `ReLU`	$k=3, s=2, p=1, op=1$	$(B, 128, 128, 128)$	转置卷积放大 2 倍分辨率
上采样层 2	`ConvTranspose2d(128, 64)` + `IN` + `ReLU`	$k=3, s=2, p=1, op=1$	$(B, 64, 256, 256)$	恢复256×256 分辨率
输出重建层	`ReflectionPad2d(3)` + `Conv2d(64, 3)` + `Tanh`	$k=7, s=1, p=3$(反射)	$(B, 3, 256, 256)$	输出$[-1, 1]$ 范围的生成图像

5.2 判别器 NLayerDiscriminator：70×70 PatchGAN 感受野数学推导

在 `models/networks.py` 第 336–375 行，默认判别器 `NLayerDiscriminator(n_layers=3)` 共包含 5 层4×4 卷积。利用感受野逆推公式$RF_{l-1} = (RF_l - 1) \times s_l + k_l$，从输出矩阵$(B, 1, 30, 30)$ 的单个元素（$RF_5 = 1$）逐层反推：

层号l	卷积配置	步幅s_l	正向输出空间尺寸	反向逆推到该层输入的感受野边长RF_{l-1}
第 5 层（输出）	`Conv2d(512, 1, k=4, s=1, p=1)`	1	30×30	$RF_4 = (1-1) \times 1 + 4 = 4$
第 4 层	`Conv2d(256, 512, k=4, s=1, p=1)`	1	31×31	$RF_3 = (4-1) \times 1 + 4 = 7$
第 3 层	`Conv2d(128, 256, k=4, s=2, p=1)`	2	32×32	$RF_2 = (7-1) \times 2 + 4 = 16$
第 2 层	`Conv2d(64, 128, k=4, s=2, p=1)`	2	64×64	$RF_1 = (16-1) \times 2 + 4 = 34$
第 1 层（输入）	`Conv2d(3, 64, k=4, s=2, p=1)`	2	128×128（输入256×256）	$RF_0 = (34-1) \times 2 + 4 = \mathbf{70}$

因此，输出矩阵30×30 中的每一个数字都精确对应原图上一个70×70 局部块（**Patch**）的真假打分！

5.3 训练循环（train.py）、train_log 诊断与微调推理（finetune/ & test.py）

1. 真实训练日志 **train_log** 的纳什均衡数值验证：在 train_log 中：

   ```
   (epoch: 1, iters: 100, time: 0.107, data: 0.016) D_A: 0.306 G_A: 0.554 cycle_A: 3.674 idt_A: 1.917 D_B: 0.2
   ```

   当博弈趋近纳什均衡点$D^*(x)=0.5$（即白板三的决策阈值$P_{\text{Threshold}}=0.5$）时，判别器理论损失为$\mathcal{L}_D^*=\frac{1}{2}[(0.5-1)^2+(0.5-0)^2]=\mathbf{0.25}$，生成器对抗损失为$\mathcal{L}_G^*=(0.5-1)^2=\mathbf{0.25}$；而 cycle_A, cycle_B 与 idt_A, idt_B 则随训练轮数单调稳步下降。

2. 预训练微调与算力加速（**finetune/fast_finetune.py**）：结合白板三的 pre-train model 思想，支持加载预训练权重并启用 **AMP** 自动混合精度（**autocast + GradScaler** 动态损失缩放防 **FP16** 梯度下溢）、梯度累积（模拟大 **Batch**）与梯度检查点（以重计算换显存）。

3. 测试推理决策（**test.py & models/test_model.py**）：对应数据科学流程的 **Decision Making** 与机器学习流程的 **Prediction / Inference**：仅加载单向预训练生成器 netG_A，在 model.eval() 与 torch.no_grad() 下对 testA 中的512×512自然风景照片做前向转换，经 tensor2im 还原为$[0,255]$的中式花鸟画保存至 results/。

附录：课堂白板公式、GAN 公式与项目命令速查表

机器学习标准流程步骤	白板一：线性回归（最小二乘 LS）	白板二 & 三：逻辑回归（MLE）	原始 GAN（Vanilla GAN）	Chinese_Flower（LSGAN + CycleGAN）
Step 1: 数据集与空间	$X\in\mathbb{R}^{k\times n}, y\in\mathbb{R}^k$	$X\in\mathbb{R}^{n\times k}, y_i\in\{0,1\}$ 或$\{-1,1\}$	真图$x\sim p_{\text{data}}(y=1)$ 假图$G(z)\sim p_g(y=0)$	无配对自然图$a\sim p_{\text{data}}(a)$ 无配对国画$b\sim p_{\text{data}}(b)$
Step 2: 模型参数化	$f(X)=Xw+c=\hat{y}\in\mathbb{R}^k$	$P(y=1\mid x_i)=\sigma(x_i^\top w+c)=\frac{1}{1+e^{-(x_i^\top w+c)}}$	判别器$D(x)=\sigma(f_D(x))$ 生成器$\tilde{x}=G(z)$	双生成器G_A, G_B（ResNet-9）双判别器D_A, D_B（70×70 PatchGAN）
Step 3: 似然 / 损失函数	$\min_{w,c}\|\hat{y}-y\|_2^2$	**MLE**: $\max\prod\sigma^{y_i}(1-\sigma)^{1-y_i}$ **NLL**: $\min\sum[-y_iz_i+\log(1+e^{z_i})]$	$\min_G\max_D\mathbb{E}[\log D(x)]+\mathbb{E}[\log(1-D(G(z)))]$	$\min_D\frac{1}{2}(D(x)-1)^2+\frac{1}{2}D(G(z))^2$ $\min_G(D(G(z))-1)^2+\lambda_{\text{cyc}}\mathcal{L}_{L_1}+\lambda_{\text{idt}}\mathcal{L}_{L_1}$
Step 4: 最优解与散度	正交投影$(y-A\hat{x})\perp Ax\implies\hat{x}=(A^\top A)^{-1}A^\top y$	凸优化梯度下降求$(\hat{w},\hat{c})$	$D_G^*(x)=\frac{p_{\text{data}}}{p_{\text{data}}+p_g}$ 等价于$\min\text{JSD}(p_{\text{data}}\,\|\,p_g)$	$D^*(x)=\frac{p_{\text{data}}}{p_{\text{data}}+p_g}$ 等价于$\min\chi^2_{\text{Pearson}}(p_{\text{data}}+p_g\,\|\,2p_g)$
Step 5: 推理与决策	$\hat{y}_{\text{new}}=x_{\text{new}}^\top\hat{w}+\hat{c}$	$p=\sigma(x_i^\top\hat{w}+\hat{c})\gtrless P_{\text{Threshold}}(0.5)$	均衡点$D^*(x)=0.5$，采样$\tilde{x}=G(z)$	推理$\tilde{b}=G_A(a)$，再经$\frac{\tilde{b}+1}{2}\times255$ 还原图像